50: Decompose Alphanumeric Words

This subroutine is entered by means of a single extension statement of the form

K: S50(n, Lβ_1, Lβ_2)

where n is a fixed-point expression giving the number of alphanumeric variables to be decomposed, β_1 is the variable whose content forms the first sequence of alphanumeric characters to be decomposed, and β_2 is the variable in which the first alphanumeric character is to be stored after decomposition. (The notation "L" indicates "location of" and is translated by the compiler to send the *address* of the variable to the subroutine.)

This subroutine will take the n alphanumeric words beginning with β_1, decompose each into five two-digit decimal pairs and store the (5 × n) two-digit decimal equivalents in the two low order (rightmost) positions of the locations beginning at β_2. The first eight decimal digits of each of these resulting words will be zero and the signs will be positive. In case the alphanumeric words do not contain legitimate characters, they will still be decomposed in the same fashion.

51: Combine Single Alphanumeric Characters

This subroutine is entered by means of a single extension statement of the form

K: S51(n, Lβ_1, Lβ_2)

where n is a fixed-point expression giving the number of single alphanumeric characters to be combined, β_1 is the variable whose content is the first alphanumeric character of the sequence to be combined, and β_2 is the variable in which the first alphanumeric word is to be stored after it is assembled.

This subroutine will take the n alphanumeric two-digit decimal characters stored in the two low order positions of the locations beginning with β_1, combine them into (n/5) or, if (n/5) is not integral, into ([n/5] + 1) alphanumeric words and store these resulting words in a sequence beginning at location β_2. The eight high order decimal digits of the sequence beginning at β_1 must be all zero to get correct results.

The resulting alphanumeric words are formed beginning at the high order end of the word, and if (n/5) is not integral the necessary low order digits of the last word formed are filled in with blanks. The signs of all resulting words are positive. The individual characters can have positive or negative signs before combination. In case the alphanumeric characters to be combined are not legitimate characters, they will still be combined in this manner.

REFERENCES

1. Perlis, A. J., Smith, J. W., and Van Zoeren, H. R., Internal Translator (IT), a compiler for the 650. Carnegie Institute of Technology Computation Center, January 1958.
2. UNC Computation Center, Automatic programming using the IT compiler for the Univac 1105 and IBM 650. Computation Center, University of North Carolina, Chapel Hill, 1959. →

Comment on A Paper on Parallel Processing

M. R. Nekora, *IBM Corporation, San Jose, California*

The article by Lynn Yarbrough on Parallel Processing in the October *Communications* is interesting since it attracts attention to a subject which needs to be given increased consideration. His indictment of manufacturers for failing to provide what he feels is minimal to realizing the advantages of multi-programming is not applicable to Stretch, however. It may be recalled that his specific complaint concerns the lack of protection of any program or monitor from the unpredictable actions of any other program. On page 15 of the Stretch Data Processing System Reference Manual, we read:

> Address monitoring facilities are provided . . . The upper and lower boundaries of the storage area to be defined are placed in two address boundary registers. An alarm will be given when an address falls either inside or outside the defined area, whichever is desired. Storing in protected areas is normally suppressed.

Memory protection is needed even while running debugged programs since it is an open question as to whether a program is ever completely debugged. All a programmer can say is that there are no detected errors! It might be mentioned that some program must set the address limits for the other programs and that error is possible in that regard. This is true but can be obviated by writing such a supervisory program without error in the first place.

One final point should be made. The author appears to be confusing multi-programming and multi-processing. Multi-processing is the simultaneous sharing of memory by different processors, as in the case of the Gamma 60 and as assumed by Yarbrough's proposed solution to the "problem". Multi-programming is the execution of several programs by transferring control among them, generally by using interrupts, but where one and only one program is in control at any one time.

As noted above, both techniques demand a system of memory partitioning to avoid chaos when running undebugged programs. Some multi-programming systems such as the Sage air defense program do not try to do debugging at the same time as their "production" runs. Stretch is capable of multi-programming (as are some other computers) but not multi-processing. Until most machines have memory protection, it is evident that multiprogramming/processing systems will not be foolproof.

Without itemizing the shortcomings of Yarbrough's scheme for memory protection or questioning its proposed implementation (which he admits is not without problems), certainly any consideration of this kind is valuable. Continuously operating computers will probably be the next milestone along the automation path. Indicating potential pitfalls is always healthy.

3. Arden, B., and Graham, R. M., Generalized Algebraic Translator. Computation Center, University of Michigan, Ann Arbor, 1959.